

GVR Compiler

Complete Compiler Implementation with Automated Testing Framework

1 Executive Summary

GVR is a complete compiler that translates high-level source code into x86 32-bit assembly language. The project includes a comprehensive automated testing framework that validates compiler output through compilation, assembly, linking, and execution verification. This document provides complete documentation for building, using, and testing the compiler.

2 Overview

2.1 Key Features

- Complete compilation pipeline from source to executable
- Target architecture: x86 32-bit (IA-32)
- AT&T syntax assembly generation
- Automated end-to-end testing framework
- Comprehensive error reporting and diagnostics
- Makefile-based build automation
- Support for interactive programs with input/output

2.2 Project Structure

Component	Description
src/	Compiler source code files
include/	Header files and declarations
gvr.out	Compiled compiler executable
Makefile	Build configuration and automation
run_tests.py	Automated test runner script
tests/	Test suite with source, expected output, and input files

3 Language Features

3.1 Data Types

- **Integers:** Signed 32-bit integers
- **Variables:** Named storage locations
- **Arrays:** Fixed-size contiguous memory blocks

3.2 Control Flow

- **Conditional:** `if`, `else` statements
- **Loops:** `while`, `for` iterations
- **Functions:** User-defined function calls

3.3 Operations

- **Arithmetic:** `+`, `-`, `*`, `/`, `%`
- **Comparison:** `==`, `!=`, `<`, `>`, `<=`, `>=`
- **Logical:** `&&`, `||`, `!`

- **Assignment:** =

3.4 Input/Output

```
read variable; // Read input
print expression; // Print output
```

4 Compiler Architecture

4.1 Compilation Stages

Stage	Description
1. Lexical Analysis	Tokenization of source code into meaningful symbols
2. Syntax Analysis	Parse tree generation and grammar validation
3. Semantic Analysis	Type checking, scope resolution, and validation
4. Intermediate Code	Three-address code (TAC) generation
5. Optimization	Constant folding, dead code elimination
6. Code Generation	x86 assembly code output

4.2 Register Usage

The compiler follows x86-32 calling conventions:

Register	Purpose
%eax	Return values, system call numbers
%ebx	System call arguments, base pointer
%ecx	Counter register for loops
%edx	Data register
%esp	Stack pointer
%ebp	Base pointer for stack frames

4.3 Assembly Generation

The compiler generates AT&T syntax assembly:

```
.section .text
.globl _start

_start:
# Program code here

# Exit system call
movl $1, %eax
movl $0, %ebx
int $0x80
```

5 Building the Compiler

5.1 Prerequisites

- **GCC/G++:** C/C++ compiler with multilib support
- **Make:** Build automation tool

- **Binutils:** GNU assembler (`as`) and linker (`ld`)
- **Python 3:** For test automation
- **32-bit Libraries:** `gcc-multilib`, `g++-multilib`

5.2 Installation (Ubuntu/Debian)

```
sudo apt-get update
sudo apt-get install build-essential gcc-multilib g++-multilib \
binutils python3
```

5.3 Build Commands

Command	Purpose
<code>make</code>	Build the compiler
<code>make clean</code>	Remove build artifacts
<code>make test</code>	Build compiler and run tests
<code>make clean && make</code>	Rebuild from scratch

6 Usage

6.1 Compiling a Program

Basic compilation workflow:

```
./gvr.out program.txt # Generates a.s
```

6.2 Assembly and Linking

```
as --32 a.s -o program.o
ld -m elf_i386 program.o -o program
./program
```

6.3 Complete Workflow

```
# 1. Write program
cat > hello.txt << 'EOF'
print 42;
EOF

# 2. Compile
./gvr.out hello.txt

# 3. Assemble and link
as --32 a.s -o hello.o
ld -m elf_i386 hello.o -o hello

# 4. Execute
./hello
```

7 Automated Testing Framework

The compiler includes a comprehensive automated testing framework that validates generated code through a complete pipeline from compilation to execution verification.

7.1 Test Pipeline

Stage	Description
1. Compile	Execute <code>./gvr.out source.txt</code> to generate assembly (<code>a.s</code>)
2. Assemble	Convert assembly to object code: <code>as --32 test.s -o test.o</code>
3. Link	Create executable: <code>ld -m elf_i386 test.o -o test</code>
4. Execute	Run executable with optional input (3-second timeout)
5. Validate	Compare actual output with expected output
6. Cleanup	Remove temporary files (<code>.s</code> , <code>.o</code> , executables)

7.2 Test File Structure

- `tests/*.txt`: Source code test cases
- `tests/*.expected`: Expected output for each test
- `tests/*.input`: Optional input data for interactive tests

7.3 Running Tests

Execute the complete test suite:

```
python3 run_tests.py
```

Example output:

```
Starting Compiler Test Suite...

Testing factorial.txt..... [ PASS ]
Testing fibonacci.txt..... [ PASS ]
Testing arithmetic.txt..... [ PASS ]

=====
Test Run Complete: 3 Passed, 0 Failed.
=====
```

7.4 Adding New Tests

1. Create source file: `tests/mytest.txt`
2. Create expected output: `tests/mytest.expected`
3. (Optional) Add input data: `tests/mytest.input`
4. Run test suite to validate

7.5 Error Diagnostics

Error Type	Diagnosis	Action
Compilation Failure	Compiler rejected input	Check syntax and semantics
Assembly Error	Invalid x86-32 assembly	Review generated assembly
Linking Error	Missing symbols	Ensure <code>_start</code> is defined
Timeout	Infinite loop (<code>i3s</code>)	Check loop termination
Output Mismatch	Incorrect result	Verify logic and format

7.6 Test Configuration

Customize test runner by modifying constants in `run_tests.py`:

Parameter	Description
COMPILER_CMD	Compiler executable path (default: <code>./gvr.out</code>)
TEST_DIR	Test directory location (default: <code>tests</code>)
COMPILER_DEFAULT_OUTPUT	Assembly output filename (default: <code>a.s</code>)

Adjust timeout value:

```
result = subprocess.run(  
    [f"./{exe_file}"],  
    timeout=3 # Adjust as needed  
)
```

8 Programming Examples

8.1 Example 1: Factorial Calculation

Source (`factorial.txt`):

```
read n;  
result = 1;  
counter = 1;  
  
while counter <= n {  
    result = result * counter;  
    counter = counter + 1;  
}  
  
print result;
```

Input (`factorial.input`): 5

Expected Output (`factorial.expected`): 120

8.2 Example 2: Fibonacci Sequence

Source (`fibonacci.txt`):

```
read n;  
a = 0;  
b = 1;  
i = 0;  
  
while i < n {  
    print a;  
    temp = a + b;  
    a = b;  
    b = temp;  
    i = i + 1;  
}
```

Input (`fibonacci.input`): 7

Expected Output (`fibonacci.expected`): 0 1 1 2 3 5 8 (one per line)

8.3 Example 3: Arithmetic Operations

Source (arithmetic.txt):

```
a = 10;
b = 3;

print a + b; // 13
print a - b; // 7
print a * b; // 30
print a / b; // 3
print a % b; // 1
```

9 Sample Input and Output

This section demonstrates the complete compilation process with actual input and output examples.

9.1 Sample Program: Sum Calculator

9.1.1 Input Source Code

File: sum.txt

```
// Simple sum calculator
read a;
read b;
sum = a + b;
print sum;
```

9.1.2 Compilation Command

```
./gvr.out sum.txt
```

9.1.3 Generated Assembly Output

File: a.s (generated by compiler)

```
.section .data
buffer: .space 12
newline: .byte 10

.section .text
.globl _start

_start:
# Read first number (a)
movl $3, %eax      # sys_read
movl $0, %ebx      # stdin
movl $buffer, %ecx # buffer
movl $12, %edx     # size
int $0x80

# Convert ASCII to integer (a)
movl $0, %eax
movl $buffer, %esi
call atoi
movl %eax, -4(%ebp) # Store a
```

```

# Read second number (b)
movl $3, %eax
movl $0, %ebx
movl $buffer, %ecx
movl $12, %edx
int $0x80

# Convert ASCII to integer (b)
movl $0, %eax
movl $buffer, %esi
call atoi
movl %eax, -8(%ebp) # Store b

# Calculate sum = a + b
movl -4(%ebp), %eax # Load a
addl -8(%ebp), %eax # Add b
movl %eax, -12(%ebp) # Store sum

# Print result
movl -12(%ebp), %eax
call print_int

# Exit
movl $1, %eax
movl $0, %ebx
int $0x80

# Helper functions (atoi, print_int)
atoi:
# ASCII to integer conversion
# Input: %esi = string pointer
# Output: %eax = integer value
# ... implementation ...
ret

print_int:
# Integer to ASCII and print
# Input: %eax = integer value
# ... implementation ...
ret

```

9.1.4 Assembly and Linking

```

# Assemble to object file
as --32 a.s -o sum.o

# Link to create executable
ld -m elf_i386 sum.o -o sum

```

9.1.5 Program Execution

```
./sum
```

User Input:

```
15
```

Program Output:

42

9.2 Sample Program: Loop Example

9.2.1 Input Source Code

File: countdown.txt

```
// Countdown from 5 to 1
counter = 5;

while counter > 0 {
    print counter;
    counter = counter - 1;
}

print 0;
```

9.2.2 Generated Assembly Output

File: a.s (excerpt showing loop structure)

```
.section .text
.globl _start

_start:
# Initialize counter = 5
movl $5, -4(%ebp)

loop_start:
# Check condition: counter > 0
movl -4(%ebp), %eax
cmpl $0, %eax
jle loop_end

# Print counter
movl -4(%ebp), %eax
call print_int

# counter = counter - 1
movl -4(%ebp), %eax
subl $1, %eax
movl %eax, -4(%ebp)

# Jump back to loop start
jmp loop_start

loop_end:
# Print 0
movl $0, %eax
call print_int

# Exit
movl $1, %eax
```



```
movl $0, %ebx
int $0x80
```

9.2.3 Program Output

```
5
4
3
2
1
0
```

9.3 Sample Program: Conditional Example

9.3.1 Input Source Code

File: max.txt

```
// Find maximum of two numbers
read x;
read y;

if x > y {
    print x;
} else {
    print y;
}
```

9.3.2 Generated Assembly Output

File: a.s (excerpt showing conditional)

```
# Read x and y (code omitted for brevity)

# Compare x > y
movl -4(%ebp), %eax # Load x
cmpl -8(%ebp), %eax # Compare with y
jle else_branch

then_branch:
# Print x
movl -4(%ebp), %eax
call print_int
jmp end_if

else_branch:
# Print y
movl -8(%ebp), %eax
call print_int

end_if:
# Exit
movl $1, %eax
movl $0, %ebx
int $0x80
```

9.3.3 Sample Execution 1

Input:

```
42
17
```

Output:

```
42
```

9.3.4 Sample Execution 2

Input:

```
15
38
```

Output:

```
38
```

9.4 Test Suite Sample Output

9.4.1 Successful Test Run

```
Starting Compiler Test Suite...

Testing arithmetic.txt..... [ PASS ]
Testing comparison.txt..... [ PASS ]
Testing factorial.txt..... [ PASS ]
Testing fibonacci.txt..... [ PASS ]
Testing loop_while.txt..... [ PASS ]
Testing conditional_if.txt..... [ PASS ]
Testing nested_loops.txt..... [ PASS ]
Testing variables.txt..... [ PASS ]

=====
Test Run Complete: 8 Passed, 0 Failed.
=====
```

9.4.2 Test Run with Failures

```
Starting Compiler Test Suite...

Testing arithmetic.txt..... [ PASS ]
Testing bad_syntax.txt..... [ FAIL ]
-> Build failed:
syntax error at line 5: expected ';' after statement

Testing infinite_loop.txt..... [ FAIL ]
-> Timed out (Infinite loop?)

Testing wrong_output.txt..... [ FAIL ]
-> Expected:
'100'
Got:
'99'
```

```
=====
Test Run Complete: 1 Passed, 3 Failed.
=====
```

10 Best Practices

10.1 Testing Strategy

- **Isolation:** Each test should verify a single language feature
- **Descriptive Naming:** Use clear filenames (e.g., `loop_while.txt`, `operator_add.txt`)
- **Edge Cases:** Include boundary conditions and error scenarios
- **Documentation:** Comment complex test cases and algorithms
- **Regression Testing:** Keep all passing tests to prevent future bugs
- **Incremental Development:** Add tests as features are implemented

10.2 Code Organization

- Track test files with compiler source in version control
- Group related tests in subdirectories
- Use consistent naming conventions
- Document non-obvious test scenarios

11 Technical Specifications

11.1 Target Platform

- **Architecture:** x86 32-bit (IA-32)
- **Operating System:** Linux
- **Assembly Format:** AT&T syntax
- **Executable Format:** ELF32
- **System Calls:** Linux syscall interface

11.2 System Call Interface

```
mov eax, 1      ; sys_exit
mov ebx, 0      ; status code
int 0x80        ; system call

mov eax, 4      ; sys_write
mov ebx, 1      ; stdout
int 0x80

mov eax, 3      ; sys_read
mov ebx, 0      ; stdin
int 0x80
```

11.3 Performance Characteristics

- **Test Timeout:** 3 seconds per test execution
- **Cleanup:** Automatic removal of intermediate files
- **Execution:** Sequential test processing

11.4 Memory Layout

```
High Address
|--- Stack (grows down)
|--- Heap (grows up)
|--- BSS (uninitialized data)
|--- Data (initialized data)
|--- Text (code)
Low Address
```

12 Troubleshooting Guide

12.1 Common Issues

- **Compilation Failure:** Review source code syntax and language specification
- **Assembly Error:** Examine generated assembly in `a.s` for syntax issues
- **Linking Error:** Ensure `_start` symbol is defined and all references resolved
- **Segmentation Fault:** Check stack management and memory access patterns
- **Timeout:** Verify loop termination conditions in source code
- **Output Mismatch:** Check whitespace, newlines, and numeric formatting

12.2 Debug Workflow

1. Run test suite to identify failures
2. Examine generated assembly for failing tests
3. Use `gdb` for runtime debugging
4. Add debug print statements to compiler
5. Verify intermediate representations (TAC)
6. Check symbol tables and type information

13 Advanced Features

13.1 Input File Support

Tests can specify input data through `.input` files. The test runner automatically:

1. Checks for matching `.input` file
2. Reads entire file contents
3. Provides data via `stdin` during execution

13.2 Custom Error Messages

The test runner provides detailed feedback:

- **Missing Files:** Alerts when `.expected` files are absent
- **Build Failures:** Shows compiler/assembler/linker errors
- **Output Differences:** Displays expected vs. actual output
- **Timeouts:** Identifies infinite loop scenarios

13.3 Color-Coded Output

- **Green:** Test passed
- **Red:** Test failed
- **Yellow:** Error details

14 Continuous Integration

14.1 CI/CD Pipeline

Integrate with continuous integration systems:

```
#!/bin/bash
set -e

echo "Building compiler..."
make clean
make

echo "Running tests..."
python3 run_tests.py

if [ $? -eq 0 ]; then
echo "All tests passed!"
exit 0
else
echo "Tests failed!"
exit 1
fi
```

14.2 GitHub Actions

```
name: GVR Compiler CI

on: [push, pull_request]

jobs:
build-and-test:
runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v2
- name: Install dependencies
run: |
sudo apt-get update
sudo apt-get install -y gcc-multilib g++-multilib
- name: Build compiler
run: make
- name: Run tests
run: python3 run_tests.py
```

15 Implementation Details

15.1 Test Execution Flow

1. Scan `tests/` directory for `.txt` files
2. For each test file:
 - (a) Check for matching `.expected` file
 - (b) Optionally load `.input` file
 - (c) Compile source with `./gvr.out`
 - (d) Assemble with GNU `as`
 - (e) Link with GNU `ld`

- (f) Execute with timeout and input
 - (g) Compare output with expected
 - (h) Clean up temporary files
3. Report summary statistics

15.2 Error Recovery

All compilation stages use exception handling:

- `CalledProcessError`: Build failures
- `TimeoutExpired`: Execution timeouts
- `finally`: Guaranteed cleanup

16 Conclusion

The GVR Compiler provides a complete solution for compiling high-level programs to x86 assembly. The integrated testing framework ensures code quality through automated validation of compiler output. By combining robust compilation with comprehensive testing, GVR enables reliable software development and facilitates compiler debugging and enhancement.

The modular architecture, detailed documentation, and extensive test suite make GVR suitable for both educational purposes and practical compiler development projects. The automated testing framework significantly reduces manual testing effort while providing detailed diagnostics for rapid debugging.

GVR Compiler Documentation • Version 1.0 • March 2026